

INTRODUCTION TO MACHINE LEARNING

MIHIR RAO

Contents

1	Introduction	3
1.1	Classification: predicting a category	3
1.2	Different classifiers fit different patterns	4
1.3	Digit images as points in a feature space	4
1.4	Training error, test error, and overfitting	5
1.5	Validation: choosing a model without using the final test	6
1.6	Supervised and unsupervised learning	7
2	Linear Classifiers and Perceptrons	8
2.1	From observations to a decision function	8
2.2	Level sets and linear boundaries	9
2.3	Vectors, dot products, and hyperplanes	10
2.4	Normals and signed distances	11
2.5	The centroid method	12
2.6	Preparing the perceptron objective	13
2.7	Risk and the optimization problem	14
3	Gradient Descent and Hard-Margin Support Vector Machines	16
3.1	Feature space and weight space	16
3.2	Gradient descent on the risk	17
3.3	One-point updates: the perceptron	19

3.4	Learning an offset and convergence	19
3.5	Maximum-margin classification	20
3.6	The hard-margin quadratic program	21
3.7	Constraints in parameter space	22
4	Soft-Margin Support Vector Machines and Features	24
4.1	Allowing margin violations	24
4.2	The soft-margin objective and the choice of C	25
4.3	Nonlinear features and parabolic lifting	27
4.4	Quadratic features	28
4.5	Higher-degree polynomial features	29
4.6	Choosing features without overfitting	30
4.7	Image features from edge orientations	31
5	Machine Learning Abstractions and Optimization	33
5.1	Four levels of a learning system	33
5.2	Unconstrained optimization and convexity	34
5.3	Numerical methods and line search	36
5.4	Smooth equality constraints	37
5.5	Linear programs and feasible regions	38
5.6	Quadratic programs and the SVM objective	39

LECTURE 1: INTRODUCTION

MACHINE LEARNING uses patterns in data to make predictions. A model describes the patterns we are willing to consider; statistics helps us reason about them; an optimization algorithm fits the model to the data. The amount and quality of the data limit what can be learned. More data can also make methods effective that worked poorly with smaller collections.

The central question is whether a learned pattern works on new examples. Remembering the examples already seen is not enough.

1.1 Classification: predicting a category

Definition 1.1 [CLASSIFICATION]. CLASSIFICATION predicts which category, or CLASS, an input belongs to. A LABEL records the known class of an example. A CLASSIFIER is a rule that assigns a predicted class to an input.

Example: *Credit card default.* For each cardholder, record the balance and annual income, together with whether the cardholder defaulted. These labeled records are TRAINING DATA. Use them to learn a rule that predicts default for a new applicant. The two classes are “default” and “no default.” In this example, balances distinguish the groups more clearly than incomes do, although the groups still overlap.

The measured inputs are FEATURES. Collecting them in a FEATURE VECTOR makes each example a point: two features give a point in a plane. A DECISION BOUNDARY separates regions where the classifier predicts different classes. It separates *predictions*; it need not separate every observed label correctly.

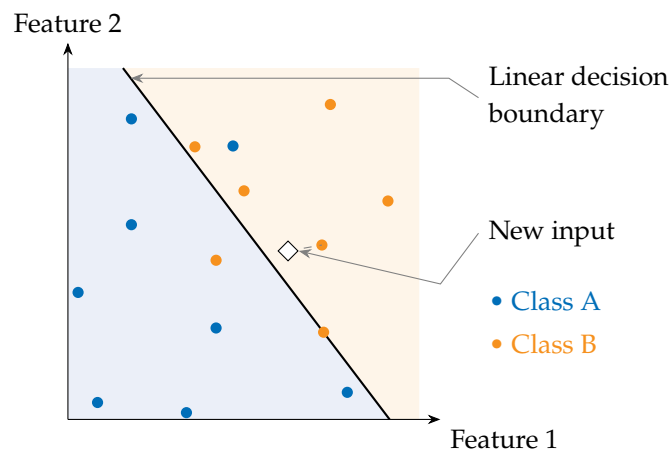


Figure 1: A schematic classification problem. The colored points have known labels; the diamond is a new input. One rule uses the nearest labeled point, while another uses the side of a straight boundary.

1.2 Different classifiers fit different patterns

The same training data can support different prediction rules.

Definition: The 1-NEAREST NEIGHBOR CLASSIFIER finds the training point closest to a new input and returns its label. The k -NEAREST NEIGHBOR CLASSIFIER instead finds the k closest training points and lets their labels vote. For two classes and odd k , one class has a majority.

At $k = 1$, the rule can respond to each individual training point. At $k = 15$, a point's label is balanced against fourteen other labels. This can give less intricate prediction regions, but may misclassify some training points. With distinct training inputs, 1-nearest neighbor classifies each training point correctly when that point itself is included among the possible neighbors: its distance to itself is zero. Conflicting labels at identical inputs would prevent this conclusion.

A LINEAR CLASSIFIER uses a straight decision boundary in two dimensions. This is a stronger restriction on the shape of the pattern. It can miss genuine nonlinear structure, but it also avoids the small islands that a nearest neighbor rule can create around individual points.

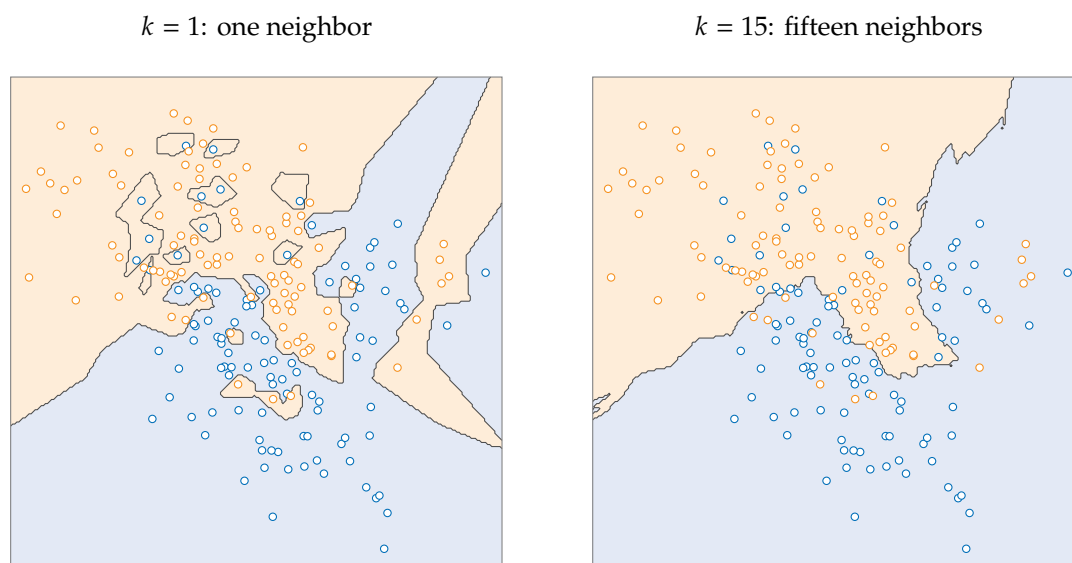


Figure 2: The same 200 labeled points classified with $k = 1$ and $k = 15$. Background colors show predictions; circles show the training labels. At $k = 1$, small regions follow individual points. At $k = 15$, votes produce broader regions, and some circles lie in a region of the other color.

Correctly classifying all training points is therefore not enough to choose a rule. A very intricate boundary may capture accidental details rather than a pattern that will persist in new data.

1.3 Digit images as points in a feature space

Example: *Handwritten 7s and 1s.* Read the sixteen pixels of each 4×4 image in a fixed order to obtain a point in $\mathbb{R}^{16}$. Labeled images of the two digits train a classifier.

4 × 4 pixel array

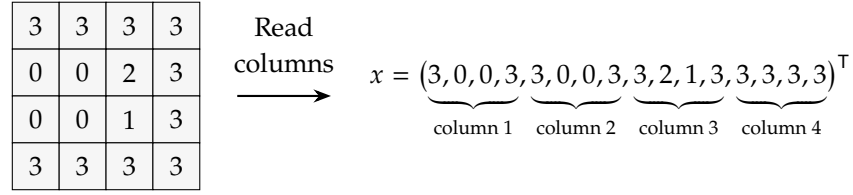


Figure 3: A 4 × 4 array becomes a sixteen-coordinate vector by reading down each column, from left to right. Training and new images must use the same coordinate order.

A **HYPERPLANE** is the higher-dimensional version of a straight line boundary. Thus a linear classifier for these images has a hyperplane as its decision boundary in sixteen-dimensional feature space.

The classification pipeline has two stages:

1. *Training*: compute feature vectors for the labeled images, then learn a classifier from the vectors and their labels.
2. *Prediction*: compute the feature vector of a new image in the same way, then evaluate the learned classifier on that vector.

1.4 Training error, test error, and overfitting

Definition 1.2 [TRAINING AND TEST ERROR]. The **TRAINING ERROR** is the fraction of training examples that the classifier labels incorrectly. The **TEST ERROR** is the fraction labeled incorrectly in a separate collection of new examples that were not used to train the classifier.

If a labeled set contains n examples, the error rate is

$$\text{error rate} = \frac{\text{number of incorrect predictions}}{n}.$$

Training error measures performance on familiar inputs. Test error checks **GENERALIZATION**: performance on new examples. The zero training error of 1-nearest neighbor says little by itself about generalization.

Definition 1.3 [OUTLIERS AND OVERFITTING]. An **OUTLIER** is a point whose label is atypical for its location, such as a borrower who appears solvent but defaults anyway. **OVERFITTING** occurs when sensitivity to outliers or other accidental patterns makes performance on new data worse.

A decision boundary should capture useful patterns. Conversely, **UNDERFITTING** occurs when the rule is too restrictive to capture the useful pattern. Very large k can ignore local distinctions; very small k can react too strongly to individual examples.

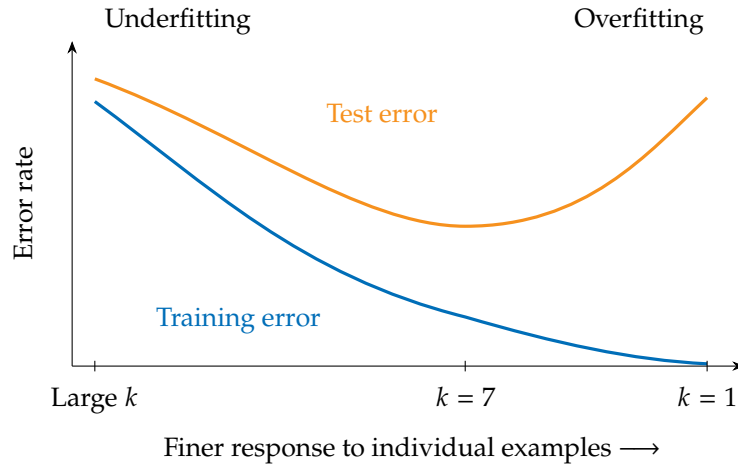


Figure 4: Schematic error trends as k decreases and the rule responds to finer details. Training error can keep falling while error on new data rises. The illustrated comparison has its best test performance near $k = 7$; this is an example, not a universal choice of k . No measured error rates are encoded by these curves.

1.5 Validation: choosing a model without using the final test

Definition: A **HYPERPARAMETER** is a setting that controls how a model is trained or how flexible its prediction rule is. The neighbor count k is one example. **VALIDATION** uses held-back labeled data to choose hyperparameters or compare models.

For a choice such as k , proceed as follows:

1. Hold back a validation set from the labeled data.
2. Train candidate classifiers with different settings, using the training set.
3. Compare their validation errors and choose the setting that performs best on the validation set.

The three sets have different jobs:

Set	Purpose
Training	Learn the model's fitted values, such as its weights.
Validation	Choose settings such as k , or choose among models.
Test	Evaluate the final choice once, after the choices are settled.

Keep the test set out of tuning. If its results influence model selection, it no longer provides the intended final check. A plot of test error against k can explain overfitting; choosing k in a new experiment requires validation data.

Prediction competitions illustrate the same issue. Repeated choices can be guided by public feedback, while a private set supplies the final score. Much stronger public results than private results can indicate overfitting to the feedback used during development.

1.6 Supervised and unsupervised learning

SUPERVISED LEARNING uses examples with target outputs. In classification, the target is a category. In REGRESSION, it is a numerical value. UNSUPERVISED LEARNING seeks structure in the inputs without supplied target labels.

Task	Example question
Classification	Is this email spam?
Regression	How likely is this patient to have cancer?
CLUSTERING	Which DNA sequences are similar to each other?
DIMENSIONALITY REDUCTION	What are common features and differences among faces?

Clustering groups similar inputs. Dimensionality reduction represents important variation with fewer coordinates. Both concern structure in the inputs rather than predicting a supplied label.

Revision. Classification predicts a category from a feature vector. A decision boundary describes where the prediction changes. Nearest neighbor rules and linear rules impose different shapes on that boundary. Generalization is the goal: training accuracy alone cannot establish it.

Common mistakes: choosing a classifier only because its training error is zero; assuming smaller k always helps; changing the pixel order between training and prediction; or using the final test set to tune a model.

Check yourself: Why can 1-nearest neighbor have zero training error and still overfit? Why does a 4×4 image have sixteen coordinates? Which set chooses k , and which set evaluates the final choice? What distinguishes supervised from unsupervised learning?

Source. Jonathan Shewchuk, University of California, Berkeley, CS 189/289A, *Introduction to Machine Learning*, Spring 2022, [Lecture 1: Introduction](#), PDF pp. 2–6. Explanatory steps are completed and figures redrawn. The neighbor comparison uses the 200 plotted points on PDF p. 4 (originally ESL, Figures 2.1–2.3); error curves are schematic. Administrative material and unrelated text within textbook crops are omitted.

LECTURE 2: LINEAR CLASSIFIERS AND PERCEPTRONS

2.1 From observations to a decision function

Suppose we have n observations, each described by d features, also called `PREDICTORS`. Some observations belong to a class C ; the others do not. Represent an observation by a feature vector $x \in \mathbb{R}^d$. Its coordinates are the input variables used for prediction.

For bank loans, the two features might be income and age. The class C contains borrowers who defaulted. A classifier uses these observations to predict whether a future borrower will default from their income and age. Different rules can produce very different boundaries.

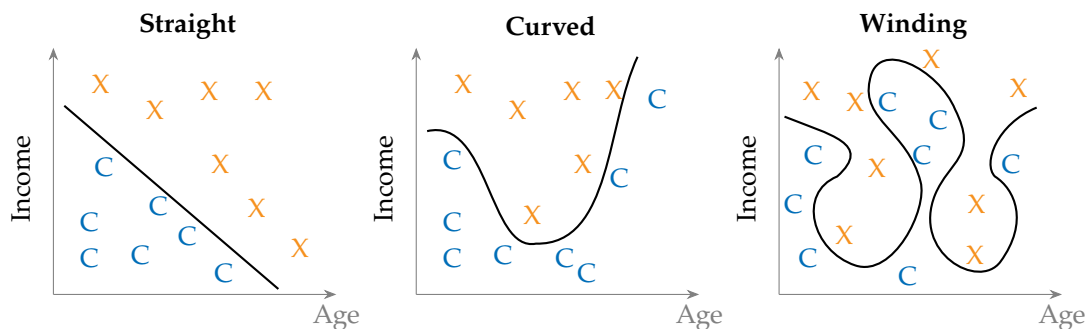


Figure 5: Schematic loan data. A straight boundary suffices on the left. The middle arrangement needs a curved boundary. The winding boundary on the right fits the observed labels closely and risks overfitting.

A boundary partitions the input space into regions assigned different classes. A complicated boundary can fit the training observations well while making poor predictions on future observations.

Definition: A `DECISION FUNCTION`, or predictor function, maps an input x to a real score $f(x)$. Under the convention used here, the classifier predicts C when $f(x) > 0$, and predicts the other class when $f(x) \leq 0$.

For a continuous score that changes sign across its zeros, the decision boundary is the zero set

$$\{x \in \mathbb{R}^d : f(x) = 0\}.$$

This is typically a surface of dimension $d - 1$: a curve in two dimensions or a surface in three dimensions. Some classifiers specify their prediction regions without using a scalar decision function.

2.2 Level sets and linear boundaries

An **ISOSURFACE**, or level set, collects the inputs with the same score. For an **ISOVALUE** c , it is $\{x : f(x) = c\}$. The decision boundary uses $c = 0$; other level sets help us see how the score changes away from it.

Example: Consider the radial score

$$f(x, y) = \sqrt{x^2 + y^2} - 3.$$

The square root is the distance from the origin. Subtracting 3 makes the score zero on the circle of radius 3, negative inside that circle, and positive outside it. Its graph is a cone. Its level set for $c \geq -3$ has radius $c + 3$; at $c = -3$ it reduces to the origin.

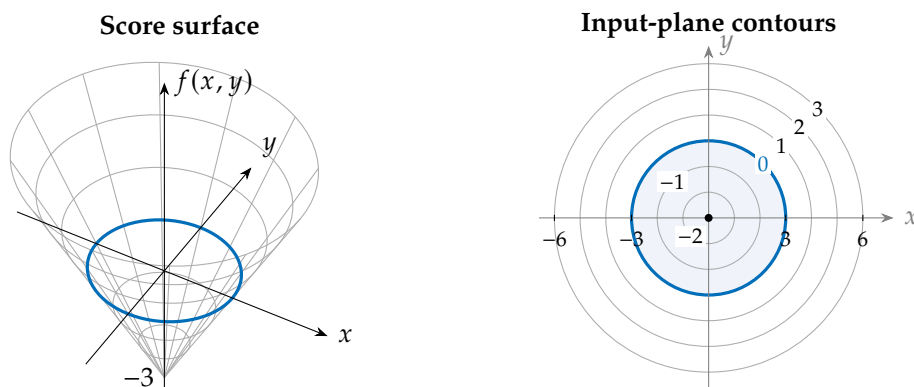


Figure 6: The same score viewed as a surface and as contours in the input plane. The blue circle is $f = 0$. The contours $f = -2, -1, 1, 2, 3$ have radii 1, 2, 4, 5, 6, respectively.

In three dimensions, the analogous score $f(x) = \|x\| - 3$ has concentric spherical level sets. The sphere of radius 3 is its decision boundary. The score is still a scalar, even though its input has three coordinates.

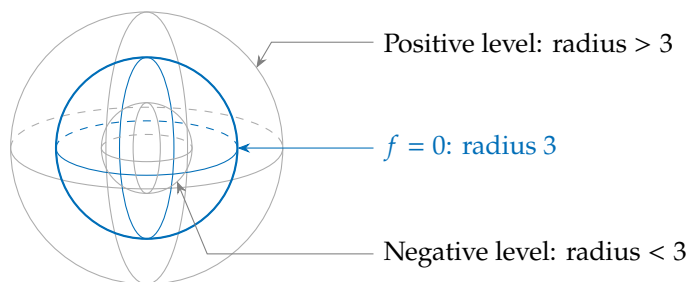


Figure 7: Spherical level sets in three-dimensional input space. Only the middle sphere, where $f = 0$, is the decision boundary.

A **LINEAR CLASSIFIER** has a flat decision boundary: a line in two dimensions, a plane in three dimensions, or a hyperplane in higher dimensions. We will describe it using $f(x) = w \cdot x + \alpha$.

2.3 Vectors, dot products, and hyperplanes

Write a vector as a column. For example, a point with five features is

$$x = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{bmatrix} = [x_1 \ x_2 \ x_3 \ x_4 \ x_5]^T.$$

The transpose T exchanges rows and columns. Lowercase Roman letters usually denote vectors; uppercase letters often denote matrices, sets, or random variables. Greek letters such as α often denote scalars. Here n counts observations, d counts features, and i, j, k are indices. These are conventions, so context still matters.

The INNER PRODUCT, or dot product, multiplies corresponding coordinates and adds the results:

$$x \cdot y = \sum_{j=1}^d x_j y_j = x^T y.$$

The EUCLIDEAN NORM measures a vector's length:

$$\|x\| = \sqrt{x \cdot x} = \sqrt{\sum_{j=1}^d x_j^2}.$$

To NORMALIZE a nonzero vector, divide it by its length. The result $x/\|x\|$ is a UNIT VECTOR, with length 1. For nonzero x, y and the angle θ between them,

$$\cos \theta = \frac{x \cdot y}{\|x\| \|y\|} = \frac{x}{\|x\|} \cdot \frac{y}{\|y\|}.$$

Thus the sign of the dot product tells us whether the angle is acute, right, or obtuse.

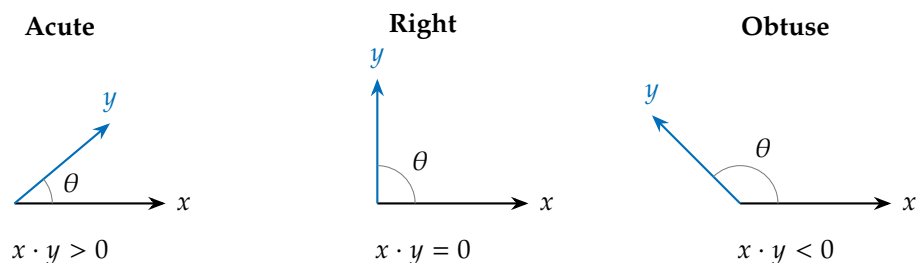


Figure 8: Dot products distinguish acute, perpendicular, and obtuse directions. The vectors must be nonzero for their angle to be defined.

Definition 2.1 [HYPERPLANE]. For $w \neq 0$, the zero set of $f(x) = w \cdot x + \alpha$ is

$$H = \{x \in \mathbb{R}^d : w \cdot x = -\alpha\}.$$

It is a flat, infinite set of dimension $d - 1$ that divides $\mathbb{R}^d$ into two half-spaces.

The score is commonly called linear. More precisely, it is **AFFINE** when the offset α is nonzero; its boundary is still a hyperplane.

2.4 Normals and signed distances

The weights w determine which direction is perpendicular to the boundary. The offset α determines its position.

Theorem 2.2 [NORMAL TO A HYPERPLANE]. If $x, y \in H = \{u : w \cdot u = -\alpha\}$, then $w \cdot (y - x) = 0$.

Proof. Subtract the two equations defining membership in H . Since $w \cdot x = w \cdot y = -\alpha$,

$$w \cdot (y - x) = w \cdot y - w \cdot x = -\alpha - (-\alpha) = 0.$$

Every displacement between points on the hyperplane is therefore perpendicular to w . □

The vector w is a **NORMAL VECTOR** of H . Parallel hyperplanes can share the same normal while having different offsets.

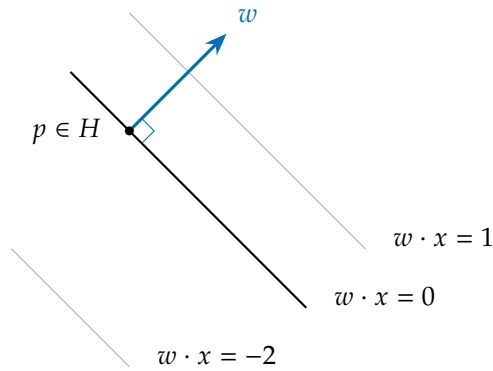


Figure 9: Parallel lines with a common normal w . Moving in the direction of w increases $w \cdot x$; moving along a line leaves it unchanged. The drawing is schematic.

The **SIGNED DISTANCE** is positive on the side toward which w points, and negative on the opposite side. For $w \neq 0$, it equals

$$s(x, H) = \frac{w \cdot x + \alpha}{\|w\|}.$$

To see this, choose a point $p \in H$ and the unit normal $u = w/\|w\|$. The perpendicular component of $x - p$ is

$$u \cdot (x - p) = \frac{w \cdot x - w \cdot p}{\|w\|} = \frac{w \cdot x + \alpha}{\|w\|}.$$

When $\|w\| = 1$, the score itself is the signed distance. Otherwise, divide both w and α by $\|w\|$ to obtain that interpretation without changing the predictions. The ordinary distance from the origin to H is $|\alpha|/\|w\|$. In particular, H passes through the origin exactly when $\alpha = 0$.

The entries of w and the offset α are the classifier's **WEIGHTS**, also called parameters or regression coefficients. Data are **LINEARLY SEPARABLE** if a hyperplane puts the two classes on opposite sides. When no such hyperplane exists, a linear classifier cannot classify all training points correctly, though it may still predict most points well.

2.5 The centroid method

A simple rule summarizes each class by its average location. Let μ_C be the mean of the training vectors in C , and μ_X the mean of those outside C . These means are the two **CENTROIDS**; both classes must contain at least one point.

The **CENTROID METHOD** places the boundary halfway between these means, perpendicular to the segment joining them. Assuming $\mu_C \neq \mu_X$, set

$$w = \mu_C - \mu_X, \quad m = \frac{\mu_C + \mu_X}{2}, \quad \alpha = -w \cdot m.$$

Its score is

$$f(x) = (\mu_C - \mu_X) \cdot x - (\mu_C - \mu_X) \cdot \frac{\mu_C + \mu_X}{2} = w \cdot (x - m).$$

The midpoint m is on the boundary because $f(m) = 0$. The normal is the direction from μ_X to μ_C , so the positive side contains μ_C .

This is also a nearest-centroid rule. Expanding the two squared distances and canceling $x \cdot x$ gives

$$\begin{aligned} \|x - \mu_X\|^2 - \|x - \mu_C\|^2 &= 2(\mu_C - \mu_X) \cdot x + \|\mu_X\|^2 - \|\mu_C\|^2 \\ &= 2w \cdot (x - m) = 2f(x). \end{aligned}$$

Thus $f(x) > 0$ means that x is closer to μ_C than to μ_X . Equal distances give score zero and use the other-class tie convention.

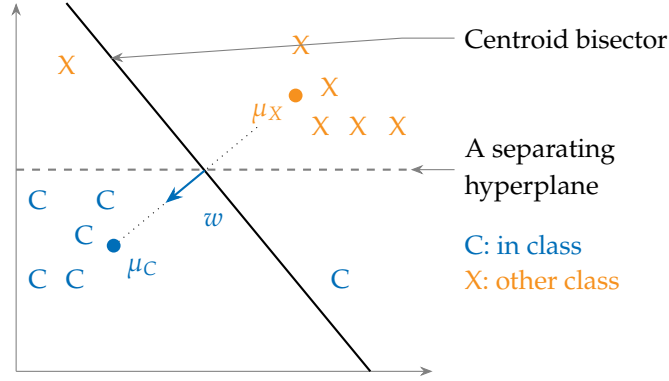


Figure 10: Centroid classification can fail on separable data. The centroid bisector misclassifies one point of each class, although the dashed horizontal line separates all points. The centroids and bisector are calculated from the schematic point positions.

Two means can hide important details of the data's arrangement. The rule can work well when the classes form two Gaussian clusters, but it need not find a separating hyperplane even when one exists. Changing only α translates the boundary without rotating it; choosing a better offset can sometimes reduce training errors. If the centroids coincide, their difference is zero and this construction does not define a separating hyperplane.

2.6 Preparing the perceptron objective

The PERCEPTRON, introduced by Frank Rosenblatt in 1957, uses numerical optimization to learn a linear classifier. Its training procedure can find a separator for linearly separable points, though it may be slow. The starting idea is to assign a penalty to violated classification constraints, then change the weights to reduce it.

Let the training vectors be $X_1, \dots, X_n \in \mathbb{R}^d$, with labels

$$y_i = \begin{cases} +1, & X_i \in C, \\ -1, & X_i \notin C. \end{cases}$$

The capital X_i identifies one observation. If observations are stored as rows of a data matrix X , its i th row is $X_i^\top$.

For now, restrict the boundary to pass through the origin: $\alpha = 0$. Then the score is $z_i = X_i \cdot w$. A positive label should have a positive score; a negative label should have a negative score. The relaxed, nonstrict constraints are

$$X_i \cdot w \geq 0 \quad (y_i = +1), \quad X_i \cdot w \leq 0 \quad (y_i = -1).$$

Multiplying by the label combines the two cases:

$$y_i X_i \cdot w \geq 0, \quad i = 1, \dots, n.$$

Each inequality is a **CONSTRAINT**. The product $y_i z_i$ checks whether the score has the desired sign. Strictly positive products put every point on the correct side; a zero product puts a point on the boundary. Also, z_i is a signed distance only if $\|w\| = 1$.

Definition: The perceptron **LOSS FUNCTION** penalizes a negative label-score product:

$$L(z, y) = \begin{cases} 0, & yz \geq 0, \\ -yz, & yz < 0, \end{cases} \quad \text{equivalently} \quad L(z, y) = \max\{0, -yz\}.$$

Correctly signed scores have zero loss. A score with the wrong sign has positive loss, which increases as it moves farther in the wrong direction.

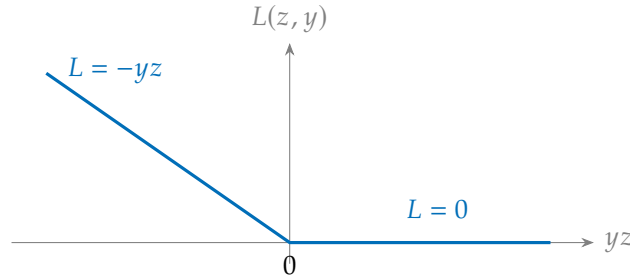


Figure 11: Loss as a function of the label-score product yz . Negative products are penalized linearly; zero and positive products have zero loss.

2.7 Risk and the optimization problem

Training combines the losses of all observations. The **RISK FUNCTION**, also called the objective or cost function, is their average:

$$R(w) = \frac{1}{n} \sum_{i=1}^n L(X_i \cdot w, y_i).$$

Let $V = \{i : y_i X_i \cdot w < 0\}$ be the indices of violated constraints. All other terms have zero loss, so

$$R(w) = -\frac{1}{n} \sum_{i \in V} y_i X_i \cdot w.$$

Each term in this sum is positive. Hence $R(w) > 0$ exactly when at least one of the nonstrict constraints is violated. The proposed optimization problem is

$$\underset{w \in \mathbb{R}^d}{\text{minimize}} \quad R(w).$$

The perceptron uses gradient-based steps to change w and reduce this objective. Here we have set up the constraints and the function to optimize.

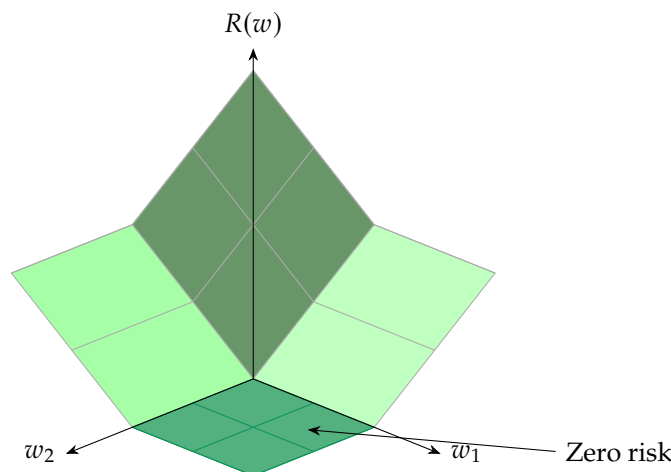


Figure 12: Schematic risk surface for two weights. Flat green regions have $R = 0$; sloping regions have positive loss. The shape depends on the training points and labels. No measured values are shown.

Note. Zero loss includes boundary ties. In particular, $w = 0$ gives $y_i X_i \cdot w = 0$ for every observation, so $R(0) = 0$, even though it does not give a useful separator. A zero score also predicts the other class under our convention, so a positive-label point on the boundary is not correctly classified. Therefore, zero risk certifies the *nonstrict inequalities*, not strict separation. Strictly positive products $y_i X_i \cdot w$ certify that all points lie on the correct sides.

Revision. The zero set of $w \cdot x + \alpha$ is a hyperplane with normal $w \neq 0$. Its signed distance is $(w \cdot x + \alpha)/\|w\|$. The centroid method uses the perpendicular bisector of the class means. The perceptron objective averages penalties for negative label-score products.

Common mistakes: treating a score as a distance without normalizing w ; forgetting the absolute value in the ordinary distance from the origin; assuming centroid classification succeeds on every separable data set; or treating zero perceptron risk as proof of strict separation.

Check yourself: Why is w perpendicular to the boundary? How does changing α move it? Why does the centroid score compare distances to the means? Why does the loss vanish at $w = 0$?

Source. Jonathan Shewchuk, University of California, Berkeley, CS 189/289A, *Introduction to Machine Learning*, Spring 2022, [Lecture 2: Linear Classifiers and Perceptrons](#), PDF pp. 1–6 (printed pp. 7–12). Figures are redrawn. The signed-distance derivation and centroid-distance identity complete the geometric arguments. The absolute value for ordinary distance and the zero-margin caveat make the stated formulas and classification convention consistent. Boundary and risk drawings are schematic.

LECTURE 3: GRADIENT DESCENT AND HARD-MARGIN SUPPORT VECTOR MACHINES

3.1 Feature space and weight space

Finding a linear classifier means finding a hyperplane that separates the labeled observations. Optimization describes the same task by searching for a point: the vector of weights defining that hyperplane. These are two different spaces, even when both have d coordinates.

In **FEATURE SPACE**, points are inputs $x \in \mathbb{R}^d$. In **WEIGHT SPACE**, points are candidate weights $w \in \mathbb{R}^d$. For now, use the origin-passing score $f(x) = w \cdot x$ and labels $y_i \in \{+1, -1\}$. The relaxed constraints are $y_i X_i \cdot w \geq 0$. We use the total perceptron loss

$$R(w) = \sum_{i=1}^n \max\{0, -y_i X_i \cdot w\} = - \sum_{i \in V} y_i X_i \cdot w, \quad V = \{i : y_i X_i \cdot w < 0\}.$$

Dropping the earlier factor $1/n$ does not change the minimizers; it only rescales the objective and its gradient.

Feature space	Weight space
Boundary $\{z : w \cdot z = 0\}$	A chosen point w
Input point $x \neq 0$	Hyperplane $\{v : x \cdot v = 0\}$

The correspondence comes from the symmetry of the dot product:

$$x \in \{z : w \cdot z = 0\} \iff w \cdot x = 0 \iff w \in \{v : x \cdot v = 0\}.$$

Thus an input lies on the classifier's boundary exactly when the weight vector lies on that input's constraint boundary. A chosen w represents a classifier; positive multiples of w represent the same classifier.

The inequality $x \cdot w \geq 0$ puts x on the side toward w in feature space. In weight space, it puts w on the side toward x . For a negative-label point, the required side reverses. All observations together restrict w to an intersection of half-spaces.

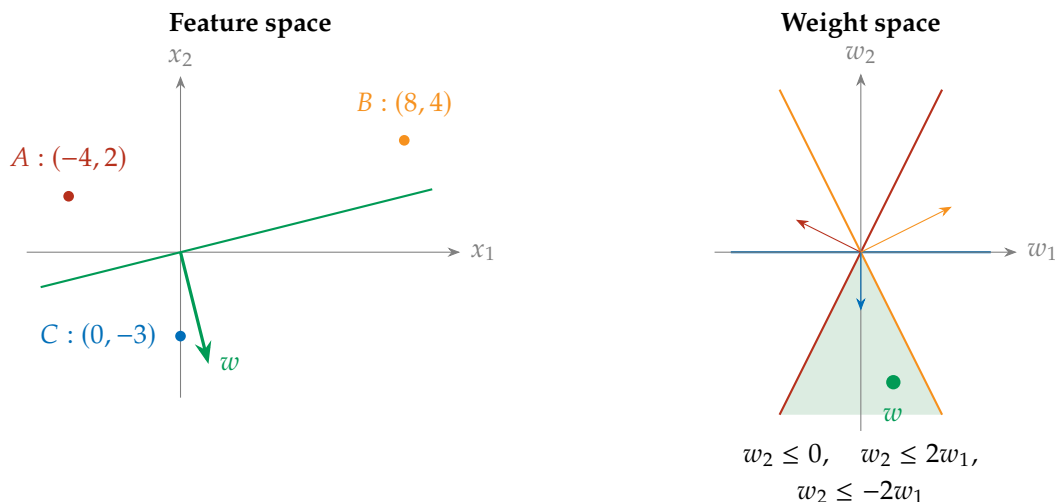


Figure 13: The same three-point arrangement in two spaces. Left: a separating line with normal $w = (1, -4)$. Right: each input defines a line through the origin. The green region satisfies all three nonstrict label constraints; its interior gives strict separation. Coordinates specify the schematic geometry.

3.2 Gradient descent on the risk

Each violated constraint contributes a linear term to R . As w crosses a constraint boundary, that term enters or leaves the sum. Consequently the risk surface consists of flat pieces joined at creases. The creases occur along the input-defined hyperplanes in weight space.

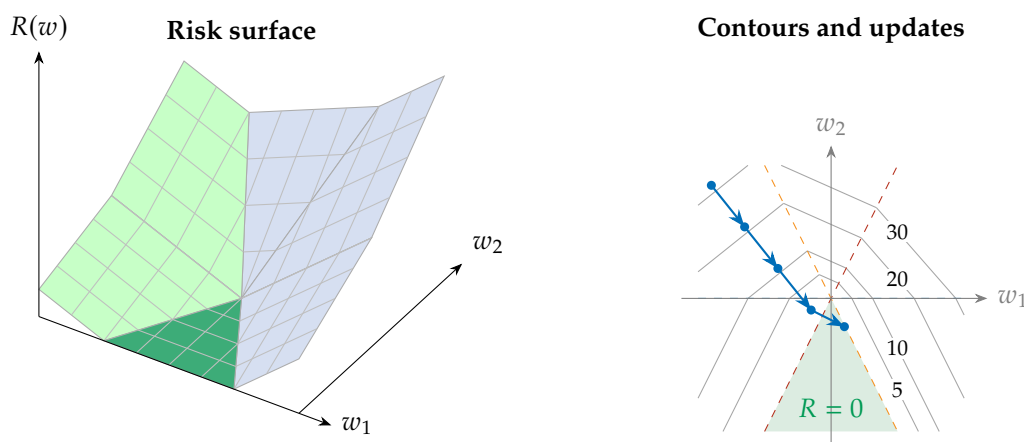


Figure 14: Risk for the three points above, viewed as a surface and as contours in weight space. The bottom wedge has zero risk. Blue arrows show batch updates toward that wedge. Contour labels are values of the total loss, not classification error rates.

GRADIENT DESCENT changes the current weights in the direction opposite the gradient. At a differentiable point, the gradient points toward the steepest local increase of the objective:

$$\nabla R(w) = \begin{bmatrix} \partial R / \partial w_1 \\ \partial R / \partial w_2 \\ \vdots \\ \partial R / \partial w_d \end{bmatrix}.$$

For a fixed vector z , differentiating $z \cdot w = \sum_j z_j w_j$ with respect to w_j gives z_j , so $\nabla_w(z \cdot w) = z$. Within a region where the violated set V is fixed,

$$\nabla R(w) = - \sum_{i \in V} y_i X_i.$$

Subtracting this gradient gives the update

$$w \leftarrow w + \epsilon \sum_{i \in V} y_i X_i,$$

where $\epsilon > 0$ is the STEP SIZE, or learning rate.

Algorithm 3.1: Batch updates for the perceptron risk

Input: Labeled vectors (X_i, y_i) ; $\epsilon > 0$; nonzero initial weights w_0 .

```

1  $w \leftarrow w_0$ 
2 while  $R(w) > 0$  do
3    $V \leftarrow \{i : y_i X_i \cdot w < 0\}$ 
4    $w \leftarrow w + \epsilon \sum_{i \in V} y_i X_i$ 
5 return  $w$ 
```

Computing all scores and summing the violated vectors takes $O(nd)$ time per iteration. A possible nonzero start is $w_0 = y_i X_i$ for any nonzero input. The learning rate is usually chosen empirically: a small step may make slow progress, while a large step can jump across a low-risk region and cause oscillation.

At a crease, the ordinary gradient need not exist. Tied points use the inactive, zero-loss branch. The batch procedure stops at nonstrict feasibility; $R = 0$ does not certify strict separation.

3.3 One-point updates: the perceptron

Instead of processing every violation at once, update using one observation. This is the STOCHASTIC GRADIENT DESCENT idea: use the gradient of one example's loss rather than the full sum. For a negative label-score product,

$$\nabla_w L(X_i \cdot w, y_i) = -y_i X_i, \quad w \leftarrow w + \epsilon y_i X_i.$$

The resulting procedure is the perceptron learning algorithm. Each weight update takes $O(d)$ time, excluding the search for a point that needs an update. A full scan to find such a point can still cost $O(nd)$.

For strict separation, treat boundary ties as unfinished as well as negative products. At a tie, the update selects the penalized branch's one-sided derivative. This also permits the convenient start $w = 0$.

Algorithm 3.2: Perceptron with boundary ties included

Input: Labeled vectors (X_i, y_i) ; fixed $\epsilon > 0$.

```

1  $w \leftarrow 0$ 
2 while there exists  $i$  with  $y_i X_i \cdot w \leq 0$  do
3   | choose an index  $i$  with  $y_i X_i \cdot w \leq 0$ 
4   |  $w \leftarrow w + \epsilon y_i X_i$ 
5 return  $w$ 
```

The correction moves toward a positive-label input and away from a negative-label input. For the selected point, its label-score product increases by

$$y_i X_i \cdot (w + \epsilon y_i X_i) - y_i X_i \cdot w = \epsilon \|X_i\|^2,$$

because $y_i^2 = 1$. Other points' scores can improve or worsen, so one correction need not finish the training problem.

3.4 Learning an offset and convergence

An origin-passing boundary is restrictive. To learn a general affine score, append a constant coordinate to every input:

$$\tilde{x} = \begin{bmatrix} x \\ 1 \end{bmatrix}, \quad \tilde{w} = \begin{bmatrix} w \\ \alpha \end{bmatrix}, \quad \tilde{w} \cdot \tilde{x} = w \cdot x + \alpha.$$

For a two-feature input, this reads

$$\begin{bmatrix} w_1 & w_2 & \alpha \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ 1 \end{bmatrix} = w_1 x_1 + w_2 x_2 + \alpha.$$

The augmented inputs lie on the plane $x_{d+1} = 1$ in $\mathbb{R}^{d+1}$. An origin-passing hyperplane there gives an affine boundary on this plane. Applying the perceptron to $(\tilde{X}_i, y_i)$ updates w and α together.

Rosenblatt's 1957 perceptron was designed for hardware image recognition. The Mark I Perceptron Machine processed 20×20 pixel images and used electric motors to adjust weights. The learning rule is also an **ONLINE ALGORITHM**: new observations can be added while updates continue, without restarting the weights.

For a separating hyperplane, its **MARGIN** is the distance to the nearest training point. A larger margin means a wider empty gap between the two classes. This distance also controls the perceptron's update bound.

Theorem 3.1 [PERCEPTRON CONVERGENCE]. *Suppose there is a unit vector u with $y_i X_i \cdot u \geq \gamma > 0$ for every i , and let $r = \max_i \|X_i\|$. Starting at zero, the tie-inclusive perceptron with a fixed positive step size finds a strict separator in $O(r^2/\gamma^2)$ updates.*

The statement requires strict linear separability in the representation being used. For an affine classifier, apply it to the augmented inputs; their lengths and margin are measured in the augmented space. It is not a guarantee for arbitrary stochastic gradient methods or nonseparable data. With a fixed nonzero initialization, the learning rate can affect the update count. With zero initialization and the same sequence of selected points, changing a constant learning rate only rescales the weights.

3.5 Maximum-margin classification

The perceptron finds a separator but does not choose the one with the widest gap. A **MAXIMUM-MARGIN CLASSIFIER** explicitly maximizes the distance from the boundary to the closest training observation. Assume that both classes are nonempty and strictly linearly separable.

For $f(x) = w \cdot x + \alpha$ with $w \neq 0$, the ordinary distance from X_i to the boundary is

$$\frac{|w \cdot X_i + \alpha|}{\|w\|}.$$

For a correctly separated point, this is also $y_i(w \cdot X_i + \alpha)/\|w\|$. We require

$$y_i(w \cdot X_i + \alpha) \geq 1, \quad i = 1, \dots, n.$$

Positive-label points must have score at least 1; negative-label points must have score at most -1 . Using 1 in place of 0 prevents the zero-weight solution when both classes are present.

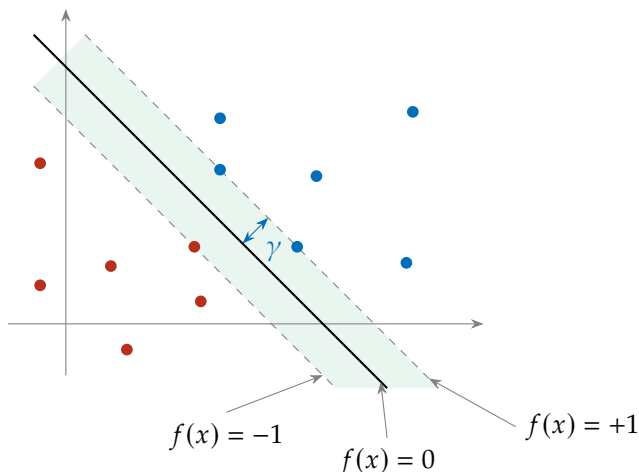


Figure 15: Blue points are in C ; red points are in the other class. A separating boundary lies halfway between the score levels -1 and $+1$. The closest points touch these two lines; the open slab between them contains no training point. The drawing is schematic.

The constraints imply the margin bound

$$\gamma(w, \alpha) = \min_i \frac{|w \cdot X_i + \alpha|}{\|w\|} \geq \frac{1}{\|w\|}.$$

Why does minimizing the norm maximize the actual margin, rather than just this lower bound? Start with any strict separator and let $\beta = \min_i y_i(w \cdot X_i + \alpha) > 0$. Dividing both w and α by β leaves the boundary unchanged and makes the smallest label-score product exactly 1. Under this normalization, $\gamma = 1/\|w\|$. Every strict separator can be represented this way.

3.6 The hard-margin quadratic program

The maximum-margin problem therefore becomes

$$\begin{aligned} & \underset{w \in \mathbb{R}^d, \alpha \in \mathbb{R}}{\text{minimize}} && \|w\|^2 \\ & \text{subject to} && y_i(X_i \cdot w + \alpha) \geq 1, \quad i = 1, \dots, n. \end{aligned}$$

This is a **QUADRATIC PROGRAM**: the objective is quadratic and the constraints are linear. It has $d + 1$ variables and n constraints. Minimizing $\|w\|^2$ and minimizing $\|w\|$ give the same minimizers, but the squared norm is smooth everywhere, including at $w = 0$. The offset α is a variable in the constraints; it is not part of the squared-norm objective.

For finite, strictly separable data with both classes present, this unique solution defines a **HARD-MARGIN SUPPORT VECTOR MACHINE (SVM)**. Nonseparable data make the constraints infeasible.

Proposition 3.2 [MARGIN AT THE OPTIMUM]. *At the optimum, at least one constraint is an equality. Hence the margin is exactly $1/\|w\|$, and the empty slab between scores -1 and $+1$ has width $2/\|w\|$.*

Proof. If every label-score product exceeded 1, their finite minimum would be some $\beta > 1$. Replacing (w, α) by $(w/\beta, \alpha/\beta)$ would keep every constraint satisfied and reduce the objective from $\|w\|^2$ to $\|w\|^2/\beta^2$. This contradicts optimality. Thus the minimum product is 1, which gives the stated distance and slab width. $\square$

3.7 Constraints in parameter space

With an offset, the parameter space has coordinates (w_1, w_2, α) for two-feature inputs. Each training point gives a plane bounding a half-space. In the three-point arrangement used earlier,

$$X_C = (0, -3), \quad X_A = (-4, 2), \quad X_B = (8, 4),$$

with labels $+1, -1, -1$, the constraints are

$$\alpha \geq 1 + 3w_2, \quad \alpha \leq -1 + 4w_1 - 2w_2, \quad \alpha \leq -1 - 8w_1 - 4w_2.$$

The feasible parameters lie above the first plane and below the other two. Their optimal intersection is

$$(w_1, w_2, \alpha) = \left(\frac{1}{17}, -\frac{6}{17}, -\frac{1}{17} \right).$$

Substitution gives scores $+1, -1, -1$ for the three inputs, so all three constraints are equalities.

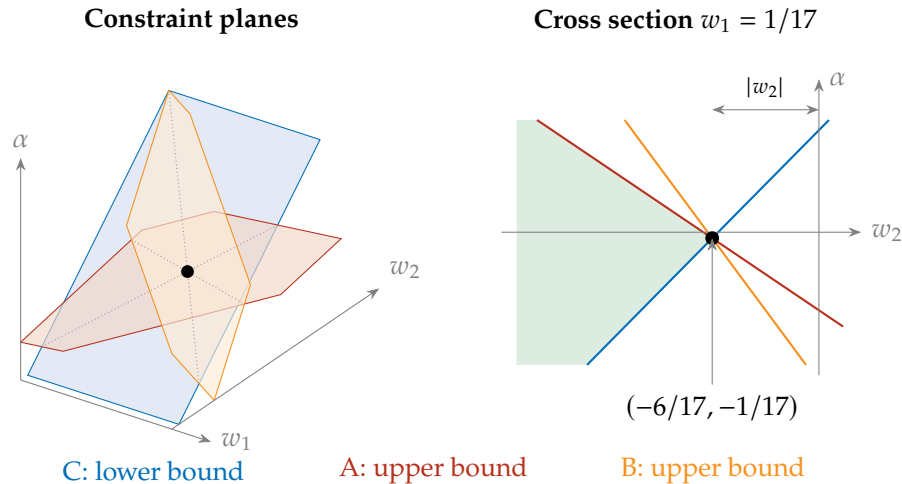


Figure 16: The three constraint planes and the cross section $w_1 = 1/17$. The green region is feasible, and the black point is the optimum. In this cross section, minimizing $w_1^2 + w_2^2$ minimizes horizontal distance to the α axis, not distance to the origin in the (w_2, α) plane.

Revision. An input point becomes a constraint hyperplane in weight space. Batch descent sums all violated vectors; the perceptron uses one point per update. Appending a constant coordinate lets the same rule learn an offset. A hard-margin SVM minimizes $\|w\|^2$ under unit label-score constraints, giving margin $1/\|w\|$ at the optimum.

Common mistakes: confusing input coordinates with weights; omitting boundary ties from a strict-separation stopping rule; counting the search for a violated point as part of an $O(d)$ update; penalizing α in the SVM norm; or confusing margin with the full slab width.

Check yourself: Why is the batch update a sum of $y_i X_i$? How does the extra coordinate update the offset? What assumptions does perceptron convergence need? Why must an optimal SVM constraint be tight, and why is its slab twice the margin?

Source. Jonathan Shewchuk, University of California, Berkeley, CS 189/289A, *Introduction to Machine Learning*, Spring 2022, [Lecture 3: Perceptron Learning; Maximum Margin Classifiers](#), PDF pp. 1–5 (printed pp. 13–17). Figures are redrawn; the three-point coordinates specify the supplied geometric arrangement and recover its $w_1 = 1/17$ cross section. The perceptron includes boundary ties and zero initialization to make the strict-separation convergence statement precise. The normalization argument and tight-constraint proof complete the margin calculation. Parameter-space distances are clarified because the objective excludes the offset. The historical photograph and press-conference quotation are omitted.

LECTURE 4: SOFT-MARGIN SUPPORT VECTOR MACHINES AND FEATURES

4.1 Allowing margin violations

A hard-margin SVM requires every training observation to be correctly classified and outside the open margin slab. This causes two problems: nonseparable data make the constraints infeasible, and one unusual observation can move the fitted boundary substantially.

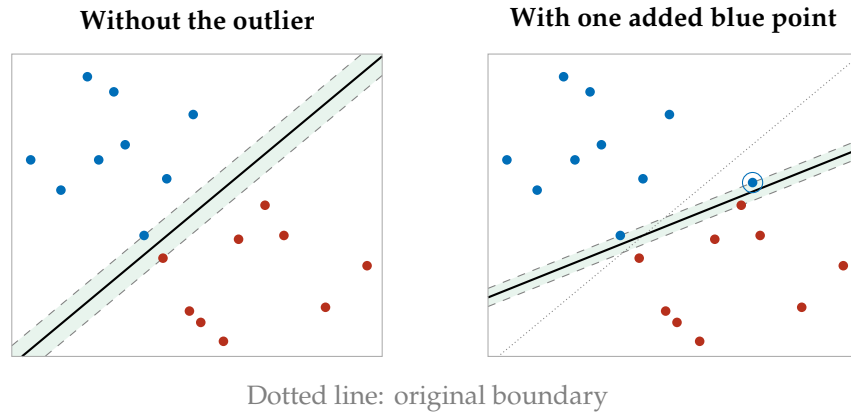


Figure 17: Schematic redraw of the outlier arrangement. The circled blue point forces a substantial change in the hard-margin boundary. The dotted line on the right shows the original boundary; the pale bands show the fitted margin slabs.

The idea of a **SOFT-MARGIN SUPPORT VECTOR MACHINE**, also called a support vector classifier, is to permit violations and charge for them. Introduce a nonnegative **SLACK VARIABLE** ξ_i for each observation. With score $f(x) = w \cdot x + \alpha$, replace the hard-margin constraint by

$$y_i f(X_i) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

Slack lowers the score required of a point. It can allow a correctly classified point inside the slab, or even a misclassified point on the wrong side of the boundary.

Write $m_i = y_i f(X_i)$ for the label-score product. For fixed (w, α) , the smallest allowed slack is

$$\xi_i = \max\{0, 1 - m_i\}.$$

Once slack has a positive cost, choosing anything larger wastes objective value. Thus the following interpretations apply to the optimal slacks:

Label-score product	Slack	Position
$m_i \geq 1$	$\xi_i = 0$	On or beyond its margin plane
$0 < m_i < 1$	$0 < \xi_i < 1$	Correct side, inside the slab
$m_i = 0$	$\xi_i = 1$	On the decision boundary
$m_i < 0$	$\xi_i > 1$	Wrong side of the boundary

For $w \neq 0$, the parallel score planes $f = 0$ and $f = \pm 1$ are $1/\|w\|$ apart. For a violating point, $\xi_i/\|w\|$ is its perpendicular shortfall from its class's margin plane. A misclassified point's shortfall extends across the decision boundary.

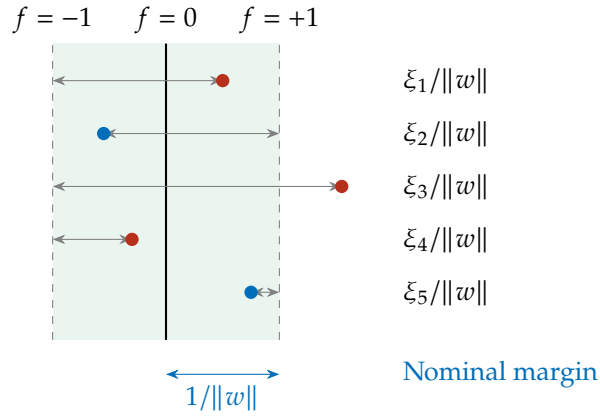


Figure 18: Five margin violations viewed along the normal direction. Blue points have positive labels; red points have negative labels. Each arrow runs from a point to its required margin plane. Rows are separated for readability; arrow lengths represent $\xi_i/\|w\|$, rather than distance to $f = 0$.

For a soft-margin classifier, *margin* means the nominal distance $1/\|w\|$ when $w \neq 0$. It is no longer the distance from the boundary to the nearest observation: points may lie anywhere inside the slab. The full nominal slab width is $2/\|w\|$.

4.2 The soft-margin objective and the choice of C

Allowing slack without charging for it would remove the incentive to fit the labels. The soft-margin optimization problem balances a small weight norm against the total amount of slack:

$$\begin{aligned}
 & \underset{w, \alpha, \xi_1, \dots, \xi_n}{\text{minimize}} && \|w\|^2 + C \sum_{i=1}^n \xi_i \\
 & \text{subject to} && y_i(X_i \cdot w + \alpha) \geq 1 - \xi_i, \quad i = 1, \dots, n, \\
 & && \xi_i \geq 0, \quad i = 1, \dots, n.
 \end{aligned}$$

This is a quadratic program with $d + n + 1$ variables and $2n$ linear constraints. The offset α remains unpenalized, and slack accommodates nonseparable labels.

The scalar $C > 0$ is a REGULARIZATION HYPERPARAMETER. It is the price per unit of slack in this objective. A small C makes violations relatively cheap, encouraging a smaller $\|w\|$ and a wider nominal margin. A large C makes violations expensive, encouraging small slack, even if fitting unusual observations narrows the margin.

	Small C	Large C
Main emphasis	Small weight norm	Small total slack
Typical risk	Underfitting	Overfitting
Outliers	More readily tolerated	Can have more influence
Nonlinear boundary	Often less flexible	Can follow the data closely

These are tendencies, rather than guarantees about test error. A linear boundary remains a hyperplane at every C ; changes in curvature require nonlinear features. Choose C by validation, rather than by training accuracy alone.

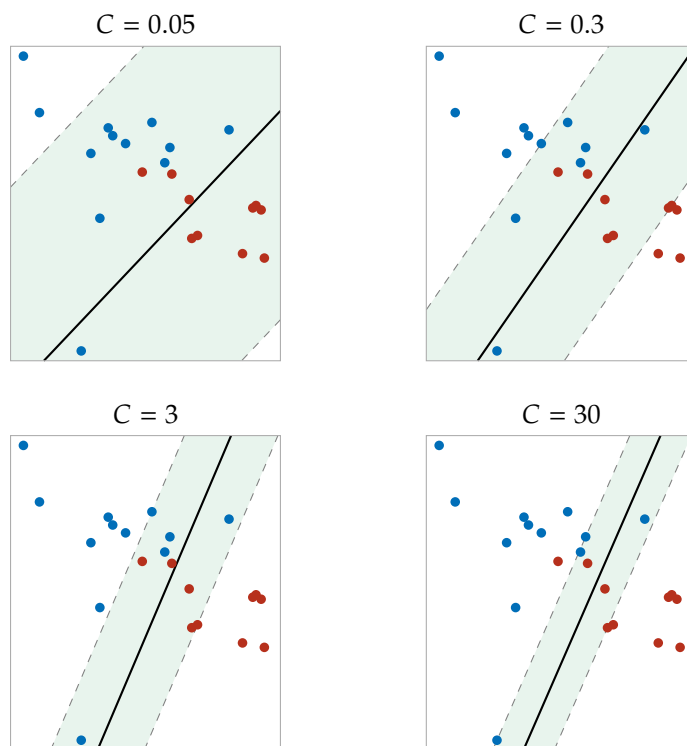


Figure 19: Increasing C makes slack more costly and narrows the fitted nominal slab in this schematic reconstruction. The same points appear in every panel. Solid lines are decision boundaries; dashed lines are margin planes. The point positions illustrate the geometry.

Think of slack as permission purchased for a point to violate its margin constraint. Greater penetration costs more. A small C accepts that cost more readily to obtain a wider margin. For separable data, the large- C limit recovers the hard-margin solution. For nonseparable data, making C large cannot make the hard-margin constraints feasible.

4.3 Nonlinear features and parabolic lifting

A hyperplane cannot separate every useful arrangement of input points. Instead of changing the linear training method, transform the inputs. A FEATURE MAP Φ computes new coordinates from x ; the classifier learns a linear score in those coordinates:

$$f(x) = w \cdot \Phi(x) + \alpha.$$

If the new coordinates depend nonlinearly on x , the boundary in the original input space can be curved. This idea also applies to classifiers that are already nonlinear.

The PARABOLIC LIFTING MAP adds one coordinate:

$$\Phi : \mathbb{R}^d \longrightarrow \mathbb{R}^{d+1}, \quad \Phi(x) = \begin{bmatrix} x \\ \|x\|^2 \end{bmatrix}.$$

The lifted points lie on the paraboloid $x_{d+1} = \|x\|^2$. Although the added feature is determined by the original coordinates, it expands the possible decision boundaries. A plane cutting the paraboloid can induce a sphere boundary in input space.

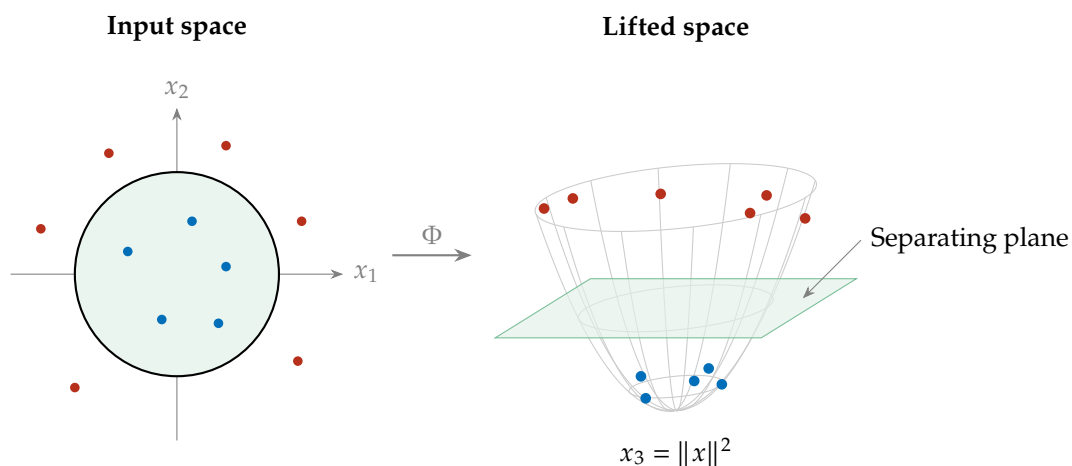


Figure 20: A circular boundary becomes a linear separation after lifting. Blue points inside the circle lie below the separating plane; red points outside it lie above. The drawing shows the geometry schematically, without changing the labels.

Theorem 4.1 [SEPARATION AFTER PARABOLIC LIFTING]. *For a finite labeled data set with both classes present, the lifted points $\Phi(X_1), \dots, \Phi(X_n)$ are strictly linearly separable if and only if the original points are strictly separable by a hypersphere or a hyperplane. Either class may occupy the inside of the sphere.*

Proof. The key is that the squared-distance formula becomes linear in the lifted coordinates. For a sphere with center c and radius ρ ,

$$\|x - c\|^2 < \rho^2 \iff \|x\|^2 - 2c \cdot x + \|c\|^2 - \rho^2 < 0.$$

Equivalently,

$$\begin{bmatrix} -2c^\top & 1 \end{bmatrix} \Phi(x) < \rho^2 - \|c\|^2.$$

Inside and outside therefore become opposite sides of a hyperplane in lifted space. An original hyperplane also lifts to a hyperplane whose last weight is zero.

Conversely, write a separating score in lifted space as

$$g(x) = a \cdot x + b\|x\|^2 + \alpha.$$

If $b = 0$, its input-space boundary is already a hyperplane. If $b \neq 0$, completing the square gives

$$\frac{g(x)}{b} = \left\| x + \frac{a}{2b} \right\|^2 - \left(\frac{\|a\|^2}{4b^2} - \frac{\alpha}{b} \right).$$

This is a sphere score with center $-a/(2b)$ and squared radius $\rho^2 = \|a\|^2/(4b^2) - \alpha/b$. Both signs occur among the observations, so $\rho^2 > 0$. Dividing by a negative b exchanges which class is inside, while preserving separation. $\square$

The hyperplane case is sometimes described as a sphere of infinite radius. Algebraically, it is precisely the case in which the weight on the squared-norm feature vanishes.

4.4 Quadratic features

Parabolic lifting gives all squared coordinates the same coefficient. To obtain axis-aligned ellipsoids and hyperboloids, give each square its own feature. In three dimensions, the boundary has the form

$$Ax_1^2 + Bx_2^2 + Cx_3^2 + Dx_1 + Ex_2 + Fx_3 + \alpha = 0.$$

Here the capital letters are scalar coefficients; this C is a coefficient, rather than the regularization parameter. Use

$$\Phi(x) = \begin{bmatrix} x_1^2 & \cdots & x_d^2 & x_1 & \cdots & x_d \end{bmatrix}^\top \in \mathbb{R}^{2d}.$$

For $d = 3$, the weights are (A, B, C, D, E, F) . The resulting score is linear in the $2d$ features even though it is quadratic in the inputs.

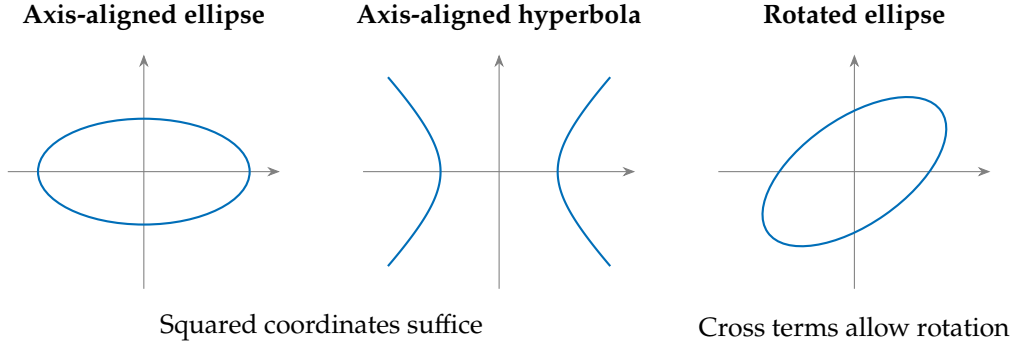


Figure 21: Quadratic decision boundaries in two dimensions, drawn schematically. Separate squared-coordinate features allow axis-aligned ellipses and hyperbolas. Cross terms allow a rotated ellipse.

To allow general orientations, add cross products of distinct input coordinates. In three dimensions, the full quadratic boundary is

$$Ax_1^2 + Bx_2^2 + Cx_3^2 + Dx_1x_2 + Ex_2x_3 + Fx_3x_1 + Gx_1 + Hx_2 + Ix_3 + \alpha = 0.$$

There are d squares, $\binom{d}{2}$ distinct cross products, and d linear terms. The feature dimension is therefore

$$d + \binom{d}{2} + d = \frac{d^2 + 3d}{2}.$$

For $d = 3$, this gives nine features. The constant term is supplied by the offset α , so it is not counted again.

The zero set of a quadratic polynomial is a **QUADRIC**; in two dimensions it is a **CONIC SECTION**. Depending on the coefficients, this includes ellipsoids, hyperboloids, and degenerate cases. The number of features now grows quadratically with the input dimension, so the extra flexibility also increases storage and computation.

4.5 Higher-degree polynomial features

A **MONOMIAL** is a product of powers of input coordinates; its total degree is the sum of their exponents. For a degree- p polynomial score, include every nonconstant monomial of total degree at most p . A cubic score in two dimensions uses

$$\Phi(x) = [x_1^3 \quad x_1^2x_2 \quad x_1x_2^2 \quad x_2^3 \quad x_1^2 \quad x_1x_2 \quad x_2^2 \quad x_1 \quad x_2]^T.$$

Each feature receives its own weight. Together with α , these weights specify every polynomial of degree at most three.

The general dimension, excluding the constant, is

$$\binom{d+p}{p} - 1.$$

For fixed p , this is $O(d^p)$ as d grows. With $d = 100$ and $p = 4$, it gives 4,598,125 features per observation. Forming and storing such vectors can be expensive. The `KERNEL TRICK` is a way to work with large feature spaces without explicitly forming these vectors; its construction is deferred.

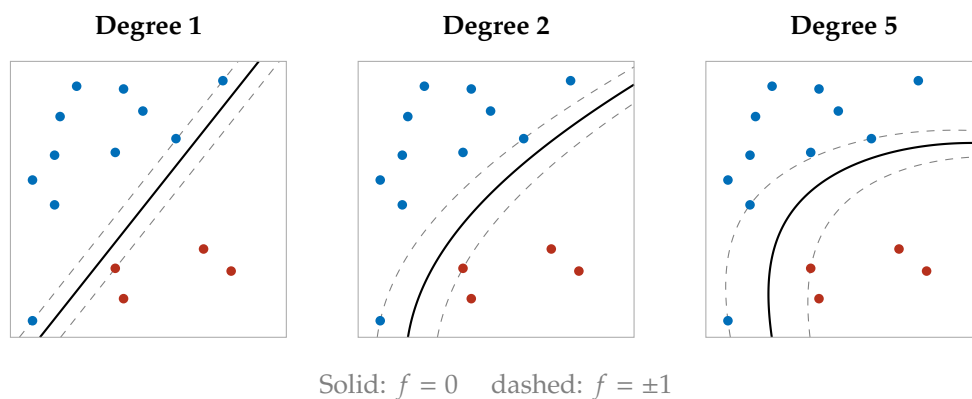


Figure 22: Schematic redraw of polynomial classifiers on the same arrangement. Higher-degree features allow curved boundaries. Each panel fits a hard-margin separator to its displayed feature map. The $f = \pm 1$ contours need not be a constant distance from the input-space boundary; SVM margin is measured in feature space.

Increasing the degree can make formerly nonseparable observations separable after lifting. It can also produce a larger margin in the chosen feature representation. Neither effect guarantees better test predictions: excessive flexibility can fit accidental details of the training sample.

4.6 Choosing features without overfitting

Polynomial degree is another hyperparameter. A degree that is too small can underfit; one that is too large can overfit. Training error alone does not select the right degree, because a more flexible family can fit the same observations more closely.

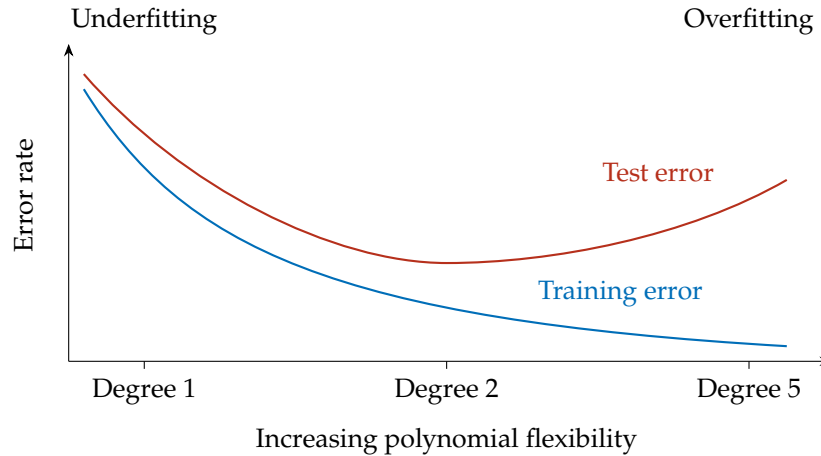


Figure 23: Illustrative training and test error as polynomial flexibility increases. Degree two gives the lowest test error in this schematic picture, while training error continues downward. The curves are conceptual, rather than measured results.

Choose the degree using validation. If polynomial features are combined with a soft-margin SVM, both degree and C require selection. Changing the degree changes the representation and can change the best C , so validate C again for each degree under consideration.

4.7 Image features from edge orientations

Features need not be polynomials. For images, useful features can describe local structure rather than raw pixel values. An EDGE DETECTOR approximates image intensity or color gradients. Examples include a tap filter, a Sobel filter, and an oriented Gaussian derivative filter. Because an image is sampled on a discrete grid, these are approximations to continuous derivatives.

Collect local line orientations into histograms, using twelve orientation bins per region. The histogram entries become the features used for classification. Regions retain local information; the bins describe which orientations are prominent there.

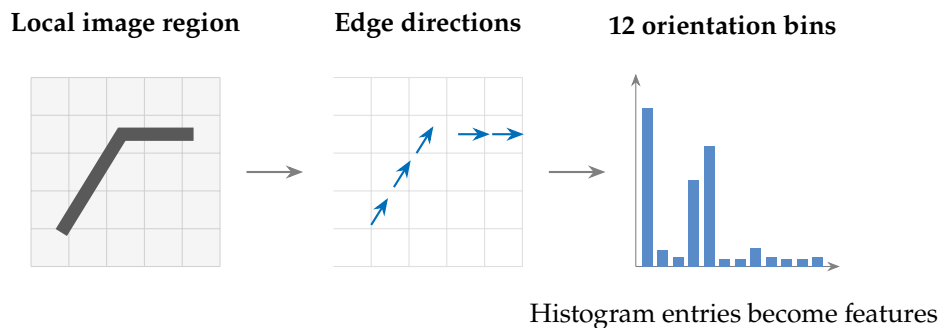


Figure 24: Local edges are summarized by an orientation histogram. Repeat the construction across image regions and collect the histogram entries into a feature vector. Bar heights illustrate the representation; they are schematic values.

In the handwriting-recognition comparison attributed to Maji and Malik (2009), a linear SVM's test error was 15.38% with raw pixels and 2.64% with histogram features. The training method remained linear; the representation made the difference. This motivates choosing features that capture structure useful for the particular task.

Revision. Slack relaxes the unit label-score constraints, and C charges for its total amount. The soft-margin distance $1/\|w\|$ is nominal, not the distance to the nearest point. A feature map makes a linear score in transformed coordinates nonlinear in the original inputs. Parabolic lifting gives sphere boundaries; squares and cross terms give quadratic boundaries. Higher-degree features add flexibility and cost, while local image histograms offer task-specific structure.

Common mistakes: treating every nonzero slack as a wrong label; reversing the effect of C in this objective; counting the constant feature twice; comparing feature-space margin with input-space distance; or selecting degree using training error alone.

Check yourself: What does $\xi_i > 1$ mean? Why is the soft-margin problem always feasible? How does a sphere become a hyperplane after lifting? Why are there $(d^2 + 3d)/2$ quadratic features? Why should C be validated again when the degree changes?

Source. Jonathan Shewchuk, University of California, Berkeley, CS 189/289A, *Introduction to Machine Learning*, Spring 2022, [Lecture 4: Soft-Margin Support Vector Machines; Features](#), PDF pp. 1–7 (printed pp. 18–24). Figures are original vector redraws; fitted panels use schematic reconstructions of the supplied point arrangements. The smallest-slack calculation, converse lifting argument, and exact monomial count complete the supplied formulas. Here C is the slack penalty coefficient, as in the displayed objective, rather than a slack budget. The image photograph is replaced by a feature-pipeline diagram; the reported image-classification errors are retained from the supplied material. Administration and future kernel details are omitted.

LECTURE 5: MACHINE LEARNING ABSTRACTIONS AND OPTIMIZATION

5.1 Four levels of a learning system

A learning system contains several choices that answer different questions. The data determine the task. The model determines which answers are permitted. The optimization problem describes how to fit that model, and the optimization algorithm computes a fit. Keeping these choices separate makes both the mathematics and the software easier to change.

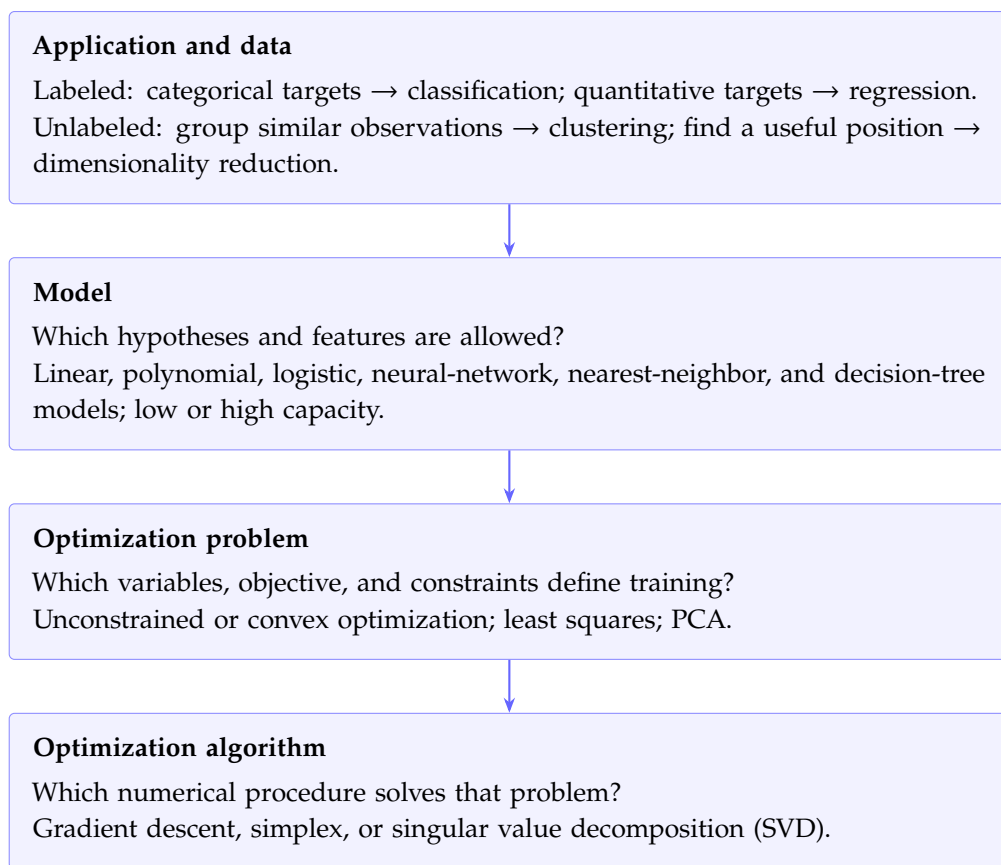


Figure 25: Four abstraction levels. Each level specifies a different part of the learning system; changing an earlier choice can require changes to the levels below it.

The **MODEL** specifies a family of hypotheses and the features they use. Its capacity affects underfitting and overfitting, while its structure affects interpretability. Useful features should reflect the data's structure. A model also needs a training problem that can be solved with the available resources.

The OPTIMIZATION PROBLEM specifies variables, an objective, and any constraints. The OPTIMIZATION ALGORITHM is the procedure used to solve that problem. A linear decision function is a model choice; minimizing a loss is a problem choice; taking gradient steps is an algorithm choice. Replacing a linear classifier with a neural network can change both the objective's geometry and the solver that is appropriate for it.

This decomposition is useful even though some learning methods do not fit it exactly. Recognizing a familiar optimization problem is a major part of translating an application into a workable training method.

5.2 Unconstrained optimization and convexity

In UNCONSTRAINED OPTIMIZATION, every parameter vector is allowed. For a continuous OBJECTIVE FUNCTION $f : \mathbb{R}^d \rightarrow \mathbb{R}$, the minimization problem is

$$\underset{w \in \mathbb{R}^d}{\text{minimize}} f(w).$$

Maximization can be handled by minimizing $-f$. A function is SMOOTH here if it is differentiable and its gradient is continuous.

Definition 5.1 [GLOBAL AND LOCAL MINIMA]. A point w^* is a GLOBAL MINIMIZER if $f(w^*) \leq f(v)$ for every $v \in \mathbb{R}^d$. It is a LOCAL MINIMIZER if there is a radius $r > 0$ such that

$$f(w^*) \leq f(v) \quad \text{whenever } \|v - w^*\| < r.$$

The minimum value is $f(w^*)$; the minimizer is the parameter vector where that value is attained.

A local minimizer has no better point in a sufficiently small neighborhood. A global minimizer has no better point anywhere. Finding a local minimum is usually more manageable than establishing global optimality for a general objective.

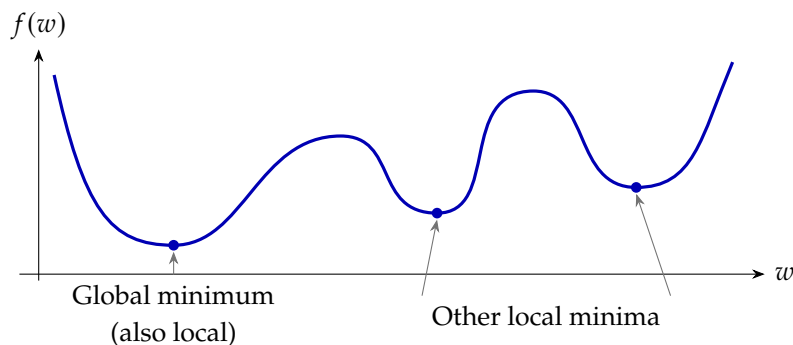


Figure 26: A schematic nonconvex objective. All three marked points are locally lowest in their neighborhoods, but only the leftmost has the lowest value over the whole domain.

Convexity gives a useful exception: a locally best point is also globally best. Geometrically, the graph never rises above a chord joining two points on it.

Definition 5.2 [CONVEX FUNCTION]. A function $f : \mathbb{R}^d \rightarrow \mathbb{R}$ is CONVEX if, for every $x, y \in \mathbb{R}^d$ and $t \in [0, 1]$,

$$f((1-t)x + ty) \leq (1-t)f(x) + tf(y).$$

The left side is the function value at a point between x and y ; the right side is the height of the chord at that point.

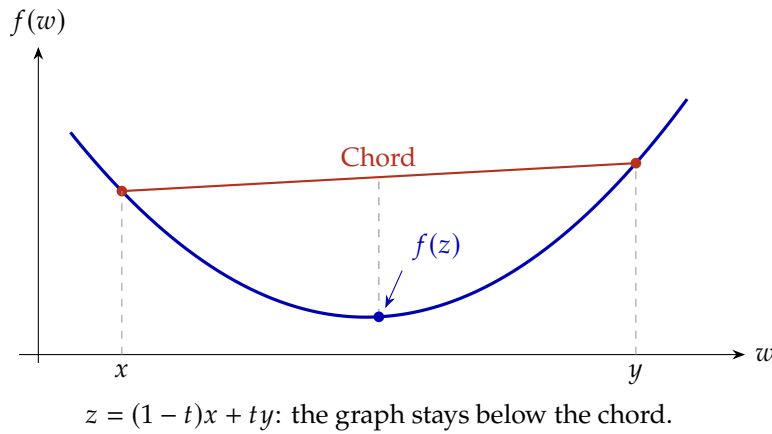


Figure 27: The convexity inequality in one dimension. At every intermediate input, the curve lies on or below the straight chord.

Proposition 5.3 [MINIMIZERS OF A CONVEX FUNCTION]. Every local minimizer of a convex function is global. If two points minimize the function, every point on their connecting segment also minimizes it, with the same objective value.

Proof. The central idea is to move a short distance toward any purportedly better point. Suppose x is a local minimizer but $f(y) < f(x)$. For a sufficiently small $t > 0$, the point $z = (1-t)x + ty$ lies in the neighborhood where x is minimal. Yet convexity gives

$$f(z) \leq (1-t)f(x) + tf(y) < f(x),$$

a contradiction. Thus x is global.

Now suppose $f(x) = f(y) = m$, the global minimum value. Convexity gives $f((1-t)x + ty) \leq m$, while global minimality gives the reverse inequality. The value is therefore exactly m along the segment. $\square$

A convex function may have no attained minimum, one minimizer, or a connected set of minimizers. On an unbounded domain, even continuity and a finite lower bound do not guarantee attainment.

The perceptron risk is convex but nonsmooth:

$$R(w) = \sum_{i=1}^n \max\{0, -y_i w \cdot X_i\}.$$

Each term is the maximum of two affine functions and is convex; adding the convexity inequalities shows that their sum is convex. Corners can occur where a term changes between its two affine pieces. Whenever the risk has several minimizers, they form a connected, equal-value set. Neural-network training objectives, in contrast, can be nonconvex and have many local minima.

5.3 Numerical methods and line search

For smooth f , the gradient points in the direction of greatest local increase. GRADIENT DESCENT moves in the opposite direction:

$$w \leftarrow w - \varepsilon \nabla f(w), \quad \varepsilon > 0.$$

The LEARNING RATE ε determines the step size. A fixed, predetermined step does not check how far along that direction the objective decreases.

Algorithm 5.1: Fixed-step gradient descent

Input: Smooth objective f , initial vector w_0 , learning rate $\varepsilon > 0$, iteration count T

```

1  $w \leftarrow w_0$ 
2 for  $t \leftarrow 1$  to  $T$  do
3    $w \leftarrow w - \varepsilon \nabla f(w)$ 
4 return  $w$ 
```

STOCHASTIC GRADIENT DESCENT instead computes an update from one training observation or a small batch at a time. This reduces the amount of data processed per update. With very large data sets, these inexpensive updates are often useful for neural-network training.

Other smooth optimization methods include NEWTON'S METHOD and NONLINEAR CONJUGATE GRADIENT. Newton's method uses the HESSIAN MATRIX, whose (j, k) entry is $\partial^2 f / \partial w_j \partial w_k$. In d dimensions it is a $d \times d$ matrix, often dense. Forming and using it can be expensive when d is large. Nonlinear conjugate gradient uses a sequence of search directions with line searches rather than only fixed steps along the negative gradient.

A LINE SEARCH chooses a step by solving a one-dimensional problem along a chosen direction p :

$$\underset{\eta \geq 0}{\text{minimize}} \phi(\eta), \quad \phi(\eta) = f(w + \eta p).$$

The goal is a suitable local minimum or decreasing step along this line, not a search over all parameter vectors. Secant and Newton–Raphson methods use derivative information for smooth line objectives. Newton–Raphson can require second derivatives, hence Hessian information. Direct methods such as golden-section search compare function values and can be used without smoothness on a bracketed interval where the function decreases to a minimum and then increases.

Note. At a nonsmooth corner, the ordinary gradient may not exist. Gradient-style methods must account for this; blindly applying the smooth formula at every point is not well defined. BFGS (Broyden–Fletcher–Goldfarb–Shanno) is another method sometimes applied to nonsmooth objectives, but smooth-method guarantees do not automatically apply. These methods seek local improvement; none of these names alone guarantees a local or global minimum for every objective and step rule.

5.4 Smooth equality constraints

A constraint restricts the allowed parameter vectors. With smooth equality constraints, the problem has the form

$$\begin{aligned} & \underset{w}{\text{minimize}} && f(w) \\ & \text{subject to} && g(w) = 0. \end{aligned}$$

The function g may be scalar or vector valued. A single equality describes a level set; several equalities require their common intersection. Improvement must respect this restricted geometry.

LAGRANGE MULTIPLIERS incorporate the equalities into a LAGRANGIAN. For $g : \mathbb{R}^d \rightarrow \mathbb{R}^m$ and $\lambda \in \mathbb{R}^m$, write

$$L(w, \lambda) = f(w) + \sum_{j=1}^m \lambda_j g_j(w).$$

At a constrained local optimum where the constraint gradients are linearly independent, the first-order conditions are

$$\nabla f(w) + \sum_{j=1}^m \lambda_j \nabla g_j(w) = 0, \quad g(w) = 0.$$

These stationarity equations balance the objective gradient against constraint normals. They do not jointly minimize L or, by themselves, prove optimality.

5.5 Linear programs and feasible regions

Linear objectives with linear constraints have a particularly clear geometry. A LINEAR PROGRAM can be written

$$\begin{aligned} & \underset{w \in \mathbb{R}^d}{\text{maximize}} && c \cdot w \\ & \text{subject to} && Aw \leq b, \end{aligned}$$

where $A \in \mathbb{R}^{n \times d}$ and $b \in \mathbb{R}^n$. The inequality is componentwise: row A_i imposes $A_i \cdot w \leq b_i$. Minimization has the same form with the objective direction reversed.

The FEASIBLE REGION is $F = \{w : Aw \leq b\}$. It is the intersection of the halfspaces allowed by all the inequalities.

Definition 5.4 [CONVEX SET]. A set P is CONVEX if, for every $p, q \in P$ and every $t \in [0, 1]$, the point $(1 - t)p + tq$ also belongs to P .

For $p, q \in F$,

$$A((1 - t)p + tq) = (1 - t)Ap + tAq \leq (1 - t)b + tb = b.$$

Thus F is convex. An intersection of finitely many halfspaces is a POLYHEDRON; a bounded one is a POLYTOPE, the higher-dimensional counterpart of a polygon. A feasible region can also be empty or unbounded.

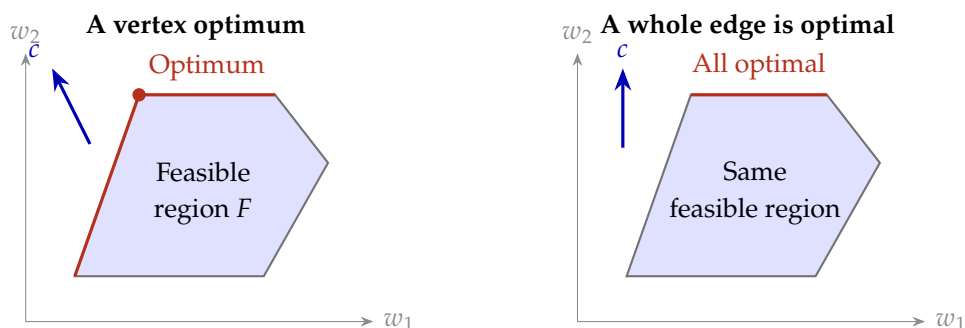


Figure 28: Schematic maximization of $c \cdot w$. Moving in the direction c favors the upper-left vertex in the first panel. Pointing straight up makes the entire top edge optimal in the second. Red edges show constraints tight at the indicated optimum or optima.

When a finite optimum is attained, it is as far as possible in the objective direction c . A constraint is ACTIVE at a point when it holds with equality, $A_i \cdot w = b_i$. The first panel has two active constraints at its optimum. An optimum need not be unique: every point on a face can have the same objective value. The optimal set is convex, since a segment between two feasible optimizers remains feasible and preserves the linear objective value.

For SVMs, active margin constraints explain why observations that touch a margin plane or violate the nominal slab can determine the fit. With positive slack cost, a soft-margin observation with nonzero slack has a tight label-score constraint $y_i(w \cdot X_i + \alpha) = 1 - \xi_i$.

Example: Finding any separating linear classifier is a feasibility problem:

$$\begin{aligned} & \underset{w, \alpha}{\text{maximize}} && 0 \\ & \text{subject to} && y_i(w \cdot X_i + \alpha) \geq 1, \quad i = 1, \dots, n. \end{aligned}$$

Every feasible pair (w, α) is equally optimal for this constant objective, and every such pair correctly classifies the observations.

Proposition 5.5 [SEPARABILITY AND FEASIBILITY]. *A finite labeled data set is strictly linearly separable if and only if the unit label-score constraints above are feasible.*

Proof. Feasibility gives $y_i(w \cdot X_i + \alpha) \geq 1 > 0$, so it gives strict separation. Conversely, for any strict separator, the finite minimum

$$m = \min_i y_i(w \cdot X_i + \alpha)$$

is positive. Replacing (w, α) by $(w/m, \alpha/m)$ makes every label-score product at least one. Hence the constraints are feasible. $\square$

The SIMPLEX METHOD, introduced by George Dantzig in 1947, searches along the polyhedron's edges from vertex to vertex. INTERIOR-POINT METHODS provide another major family of linear-programming algorithms. Which constraints become active is a central part of the search; enumerating all candidate combinations is not practical in general.

The constant objective finds a separator but does not select the one with the largest margin. Changing to a squared-norm objective leads to a quadratic program.

5.6 Quadratic programs and the SVM objective

A QUADRATIC PROGRAM minimizes a quadratic objective subject to linear constraints:

$$\begin{aligned} & \underset{w}{\text{minimize}} && f(w) = w^T Q w + c^T w \\ & \text{subject to} && A w \leq b, \end{aligned}$$

where Q is symmetric. Its curvature determines whether this is a convex optimization problem.

Definition 5.6 [POSITIVE DEFINITE AND SEMIDEFINITE MATRICES]. A symmetric matrix Q is POSITIVE DEFINITE if $v^T Q v > 0$ for every nonzero v . It is POSITIVE SEMIDEFINITE if $v^T Q v \geq 0$ for every v . It is INDEFINITE if the quadratic form takes both positive and negative values.

For $z = (1 - t)x + ty$, expansion gives

$$(1 - t)f(x) + tf(y) - f(z) = t(1 - t)(x - y)^T Q (x - y).$$

If Q is positive semidefinite, the right side is nonnegative, so f is convex. If Q is positive definite, it is positive whenever $x \neq y$ and $0 < t < 1$. This **STRICT CONVEXITY** rules out two distinct minimizers on a convex feasible region: their midpoint would have a strictly smaller value.

For a nonempty closed polyhedral feasible region and positive definite Q , the quadratic grows without bound as $\|w\| \rightarrow \infty$, so a minimum is attained and is unique. With positive semidefinite Q , an attained minimum may be unique or may have infinitely many minimizers. An indefinite Q gives a nonconvex objective; global quadratic optimization is NP-hard in general, although particular instances can be easier.

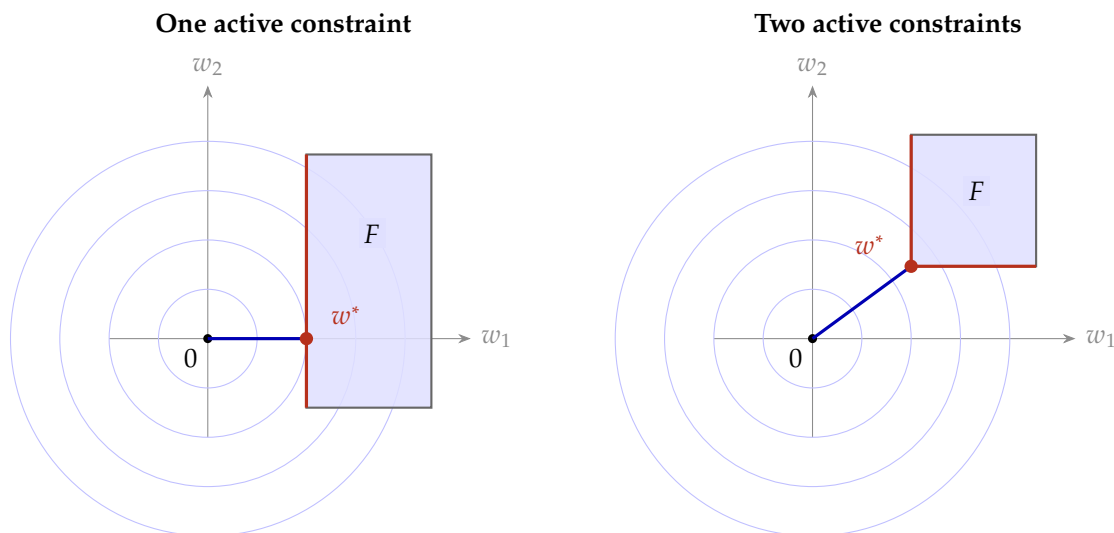


Figure 29: Schematic minimization of $\|w\|^2$ over two polygons. Circles are equal-value contours. The smallest circle meeting the feasible region touches an edge in the first panel and a vertex in the second. Red boundaries are active at w^* .

The hard-margin SVM uses this squared-norm geometry:

$$\begin{aligned} & \underset{w, \alpha}{\text{minimize}} && \|w\|^2 \\ & \text{subject to} && y_i(w \cdot X_i + \alpha) \geq 1, \quad i = 1, \dots, n. \end{aligned}$$

Its feasible region is the same one used to find any separator. The objective now chooses the smallest weight norm, hence the largest margin. The distance is measured in the w coordinates: α is not penalized. In the full variable vector (w, α) , the quadratic matrix has a zero entry in the offset direction and is positive semidefinite, rather than positive definite. Thus the positive-definite uniqueness argument cannot simply be applied to all SVM variables.

Quadratic-programming algorithms include simplex-like methods, **SEQUENTIAL MINIMAL OPTIMIZATION** (SMO, used by LIBSVM), and **COORDINATE DESCENT** (used by LIBLINEAR). The latter two exploit SVM structure. SMO changes a pair of coordinates at a time; this allows a cheap line optimization using only the relevant curvature information, rather than processing a full dense Hessian or always moving in the steepest-descent direction.

Revision. Separate the task, model, training problem, and solver. A local minimum only compares nearby points; a global minimum compares the entire domain. Convexity makes every local minimum global and makes the minimizer set convex. Linear programs optimize a linear objective over halfspaces. Convex quadratic programs add nonnegative quadratic curvature; SVMs use it to prefer a small weight norm.

Common mistakes: confusing a model with its solver; assuming convexity guarantees an attained or unique minimum; using an ordinary gradient at a corner; treating Lagrange stationarity as a proof of optimality; or assuming the SVM offset is penalized.

Check yourself: Why is a local minimum global for a convex function? What does a line search optimize? Why is a linear program's feasible region convex? How does rescaling prove the separability criterion? Why does positive definite Q imply uniqueness, while positive semidefinite Q need not?

Source. Jonathan Shewchuk, University of California, Berkeley, CS 189/289A, *Introduction to Machine Learning*, Spring 2022, [Lecture 5: Machine Learning Abstractions and Numerical Optimization](#), PDF pp. 1–5 (printed pp. 25–29). Figures are original vector redraws of the supplied hierarchy and optimization geometry. The convex-minimizer, feasibility, and quadratic-curvature arguments complete the supplied claims. Attainment, nonsmooth updates, equality-constraint stationarity, and the unpenalized SVM offset are qualified to avoid unconditional optimality or uniqueness claims. BFGS's nonsmooth qualification was checked against Lewis and Overton, [Nonsmooth Optimization via BFGS](#). Course administration is omitted.